

Guía de supervivencia práctica · NestJS

Misma temática de siempre (middlewares · usuarios y bcrypt · JWT y roles), pero ordenada para que **no te sientas solo frente al código**: primero el mapa mental, luego teoría y consignas, y abajo el tutorial con *esqueletos*, *botón de pánico* ante errores típicos y *checkpoints* antes de avanzar.

Parte I teoría completa → **Parte II** todas las consignas → **Parte III** paso a paso ejecutable.

Vas a tener dudas. Es normal. Cuando Nest parezca mágico oscuro, volvé a esta cajita: no hace falta entender TODO de memoria; hace falta saber **en qué capa** tocar cada cosa.

EL MAPA GRANDE (THE BIG PICTURE)

Un **request** no “aparece” en tu controlador: atraviesa capas. Si sabés en qué orden van, dejás de adivinar por qué algo no corre.



Versión alumnos · Un solo archivo: funciona offline. Para PDF: **Ctrl/Cmd+P → Guardar como PDF**. Podés aumentar el texto con **Modo clase** en la barra superior.

1. **Parte I** Teoría completa (middleware → usuarios/bcrypt → JWT)
2. **Parte II** Consignas del laboratorio (las tres)

3. **Parte III** Tutorial paso a paso: logger · usuarios · JWT (`#log-tutorial` · `#usr-tutorial` · `#jwt-tutorial`)

4. FAQ · tabla resumen · siguiente lectura

El laboratorio sigue en SQLite + TypeORM. Copiás `.env.example` como `.env` y definís secreto JWT real. **Primera alta = admin principal**; los siguientes registros llegan como `user` hasta que un admin ejecute `PATCH` o `PUT /users/<id>/role ( { "role": "admin" } )` con token admin. La **barra superior** ajusta tamaño de letra y pantalla completa para PDF o proyector.

PARTE I

Teoría completa

Leé **toda** la teoría antes de las consignas: aquí tenés mantra de una línea por pieza nueva y un checkpoint antes de cambiar de capítulo.

Después viene Parte III con código a completar vos.

LAS TRES PIEZAS ALREDEDOR DEL CONTROLADOR · POR QUÉ EL LOGGER PRIMERO

Viaje HTTP en Nest antes de usuarios ni JWT

Nest ejecuta código en orden aproximado: primero lo que configures como **middleware HTTP**, luego pueden encadenarse **guards**, después **interceptors**, el método del controlador, y los interceptors al salir.

La regla práctica que te salva: **cada cosa donde encaja mejor** — middleware barato sin identidad fuerte, JWT y roles donde ya conocés qué método corre (`ExecutionContext`).

1. Middleware (`NestMiddleware`)

El middleware es como el vigilante que anota cada visita antes de llamar ascensor: sólo necesita método, URL y tiempo; casi ni sabe qué método del controller va a ejecutarse.

Corre muy **temprano** sobre `req/res` de Express (lo habitual). Firma `(req, res, next)` y **debe** llamar `next()`.

Uso habitual: logs y cabecera de tiempo antes del controller.

2. Guards (`CanActivate`)

Un Guard es el portero de tu API en la puerta correcta: decide quién entra a este handler, no "a cualquier lado" del servidor.

Responden **sí/no** ante "¿entra esta petición a este *handler* concreto?". Autenticación (¿Bearer válido?) y autorización por rol típicamente viven acá.

No metas JWT suelto en middleware global si vas a usar `@Public()` o decoradores sobre rutas específicas: el guard ordena mejor con Passport.

3. Interceptors

El interceptor es el envoltorio antes y después del plato principal: bien para formato de respuesta o medición del negocio, no para fingir seguridad donde corresponda un Guard.

Envuelven entrada y salida del handler; no sustituyen al guard ni al middleware cercano al cable.

Orden práctico (resumen): petición → middlewares HTTP → guards → interceptor (entrada) → controlador → **servicio(s)** → respuesta (← interceptor salida).

Checkpoint · capítulo 1 (solo teoría middleware)

- Sin mirar código: ¿por qué el logger tiene sentido antes que validar JWT?

- ¿Qué problema aparece si el rol se decide en middleware global en lugar de guards?
- ¿Qué hace falta llamar después de registrar el log si no queremos cortar todas las rutas?

2 · Entorno, usuarios y bcrypt

TEORÍA COMPLETA

`.ENV` + PERSISTENCIA · CONTRASEÑAS FUERA DEL ZIP

Configuración (Twelve-Factor) antes de exponer rutas públicas

El `.env` es la libreta que no subís al zip: el código lee nombres de variables; los valores reales viven en tu máquina o en el servidor.

El factor III del **Twelve-Factor App**: secretos JWT, costo bcrypt, ruta SQLite... **fuera del repo**.

`.env.example` documenta nombres con valores ficticios; `.env` real → ignorado por Git.

Nest usa `ConfigModule.forRoot({ isGlobal: true })` y `ConfigService` (`getOrThrow` cuando preferís que el proceso *no arranque* con config rota).

Usuario persistente · hash con bcrypt

bcrypt no “encripta para desencriptar después”: genera un hash con sal; en login comparás con `compare` — eso endurece fuerza bruta incluso si alguien roba tu base.

Módulo usuarios típico: entidad TypeORM, servicio (register/login), roles en un mismo modelo didáctico.

bcrypt hace cara cada llamada deliberadamente. Mismo texto ⇒ hashes distintos (sal distinta). Validás solo con `bcrypt.compare(plain, hash)`. `BCRYPT_COST` aumenta ese trabajo.

Analogía breve

Un ataque masa de intentos ⇒ el costo CPU se va de las manos; por eso el curso fuerza factor alto y hashing serio antes de tirar JWT arriba.

Checkpoint · capítulo 2 (teoría entorno/usuarios)

- Si mandás tu `.env` por mail al compañero sin querer: ¿por qué puede ser grave aunque el repo compile?
- ¿Por qué exponer contraseña en texto en JSON desde el servidor es incompatible con el modelo seguridad clase?
- ¿Qué diferencia práctica existe entre aplicar bcrypt en el usuario de la BD y hacerlo “después cuando tengamos tiempo”?

¿Qué es un JWT?

Un JWT es un ticket firmado digitalmente: el servidor confía en la firma; no es un “sobre sellado” — el payload va codificado, no cifrado: no pongas contraseñas ahí.

JWT (JSON Web Token, RFC 7519): cadena **firmada** con *claims* (`sub`, expiración...). Verificás firma + tiempo. Tres partes: `HEADER.PAYLOAD.FIRMA`.

Tras login: `{ "access_token": "..." }`. Rutas protegidas: `Authorization: Bearer <token>`. Firma mala o vencida ⇒ típicamente **401**.

Autenticación vs autorización

Autenticación: ¿sos vos? · Autorización: aun siendo vos, ¿esta ruta te deja hacer esto?. **401** ante identidad floja · **403** ante permiso ausente.

Ejemplo: token válido, rol `user`, ruta sólo-`admin` ⇒ **403**.

Dónde vive JWT en Nest

Passport + estrategias es el despacho de identidad reusable: `JwtStrategy` pobla `req.user`; los `guards` sólo tienen que decidir qué hacer con ese usuario.

- **JwtService**: firmar payloads tras bcrypt ok.
- **JwtStrategy** (`passport-jwt`): validar Bearer → `req.user`.
- **JwtAuthGuard** antes que **RolesGuard** cuando dependés del mismo request autenticado.

¿Por qué ordenar el trabajo práctico?

Dependencias → `.env` → modelo usuario bcrypt que compila ⇒ luego Guards + Passport ⇒ curls públicos/admin.

Checkpoint · capítulo 3 (teoría JWT)

- ¿Por qué un token válido pero con rol insuficiente es 403 y no “otro login”?
- ¿Qué problema hay en confiar solo en el contenido visible del JWT en `jwt.io` como “prueba” de seguridad?
- ¿Para qué conviene tener **JwtAuthGuard** ordenado antes de **RolesGuard**?

Consignas prácticas (todas)

Acá están reunidas las **tres consignas** del laboratorio: logger, usuarios bcrypt y JWT con roles. Si algo no aparece como ítem marcable aquí abajo, no es requisito de entrega práctica cerrada. Después vas a la Parte III para ejecutar reproducible ordenado.

1 · Middleware y observabilidad **CONSIGNA PRÁCTICA**

Qué tenés que lograr (solo demostrable)

Log por request

En `start:dev`, cada request muestra método, URL y tiempo legible.

Cabecera medible

`curl -i ...` incluye `X-Response-Time`.

AppModule

`MiddlewareConsumer`: primero logger, después timing.

Build limpio

`npm run build` sin errores.

Opciones del lab MD: `docs/ejercicio-middlewares-alumnos.md`

Qué tenés que lograr antes de JWT

Variables reproducibles

`.env.example` + `.env` ignorado por Git · `ConfigModule.forRoot({ isGlobal: true })`.

Usuario con hash persistido

Email único · `passwordHash` (nunca plano expuesto en JSON/logs) · `role`.

Register

`bcrypt.hash` usando `BCRYPT_COST` desde env.

Login

`bcrypt.compare` · mismo mensaje ante mail inexistente o password malo.

Bootstrap admin

Tabla vacía ⇒ primer alto `admin`; siguientes registros usuario `user` hasta promoción cerrada por admin más adelante.

Dependencias de persistencia/hash

`bcrypt` instalado (+ types dev) junto al stack TypeORM acordado en el proyecto guía.

Qué tenés que lograr (auth completa)

Paquetes

`@nestjs/jwt` · Passport · passport-jwt · (+ tipos dev) donde falten.

`/auth/register` + `/auth/login`

Login devuelve `access_token` JWT (tres segmentos).

Públicos vs JWT

`/placeholder-users` público · rutas admin con Bearer admin · user sin Bearer → donde aplique JWT 401 ; user con JWT sobre ruta admin-only → 403 .

Implementación declarada

`@Roles()` + `RolesGuard` registrado · rutas típicamente `@UseGuards(JwtAuthGuard, RolesGuard)` ordenado así.

Payload

Firma JWT con claims `sub` y `role` (re-leer BD en validate recomendado según proyecto `api-c`).

README + build

`npm run build` limpio y otro equipo reproduce con tus instrucciones.

PARTE III

Explicación y tutorial paso a paso

Esta parte es taller guiado: **primero la explicación de qué hace cada archivo**, después el **código completo listo para copiar/adaptar** en tu proyecto. Siguen los errores esperables ("botón del pánico") y los checkpoints.

Orden:

1) logger y cabecera · **2)** usuarios y bcrypt · **3)** JWT y curls.

Parte III · capítulo A · logger · timing · código completo

Laboratorio 1 · Logger y cabecera X-Response-Time

Antes: teoría `#log-teoria` · consigna `#log-consigna` . **Abajo vas a tener:** para cada archivo, párrafos que dicen exactamente para qué sirve cada pieza seguidos del archivo entero funcionando — vos lo adaptás a tu ruta si tu template difiere, pero sin adivinar qué línea falta.

1. Crear `src/common/middlewares/` .
2. Dos middlewares (`NestMiddleware`) · registrar en `AppModule` (`NestModule`).
3. `npm run build` → `npm run start:dev` → `curl` y leer stdout + cabeceras.

Paso 1 de 9 · Carpeta lista

Ubicación: `src/common/middlewares/` .

Paso 2 de 9 · Logger middleware (archivo completo)

Este archivo registra cada request que pasa antes del controller. Debés obtener hora estándar con `toISOString()` ; loguear método HTTP y camino útil (`originalUrl ?? url`); y siempre llamar `next()` — si no, la cadena corta ahí mismo y Nest nunca ejecuta tus rutas.

```
import { Injectable, NestMiddleware } from '@nestjs/common';
import type { NextFunction, Request, Response } from 'express';

@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, _res: Response, next: NextFunction): void {
    const ts = new Date().toISOString();
    console.log(`[${ts}] ${req.method} ${req.originalUrl ?? req.url}`);
    next();
  }
}
```

Botón del pánico · middleware

Error típico: TypeScript marca tipos de Express (`Request` , etc.).

Frecuente: falta paquete de tipos (`@types/express`) o import path incorrecto.

Acción: guardá archivo bajo `src/` , corré `npm run build` y leé el mensaje antes de otros cambios.

Paso 3 de 9 · Timing middleware — cabecera X-Response-Time

Guardamos `Date.now()` al entrar al middleware. Como el body se escribe después de ejecutar controllers, intercalamos temporalmente la función `res.send` de Express: así calculamos milisegundos y seteamos la cabecera **antes** de que la respuesta vuelva al cliente por el mismo hueco donde Express escribe headers.

```
import { Injectable, NestMiddleware } from '@nestjs/common';
import type { NextFunction, Request, Response } from 'express';

@Injectable()
export class TimingMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction): void {
    const startAt = Date.now();
    const originalSend = res.send.bind(res);

    res.send = function (body: unknown) {
      const ms = Date.now() - startAt;
      res.setHeader('X-Response-Time', `${ms} ms`);
      return originalSend(body);
    };

    next();
  }
}
```

Paso 4 de 9 · AppModule debe implementar NestModule

Debes declarar clase `AppModule` cumpliendo `NestModule`: eso sólo quiere método `configure(consumer)`. Ahí llamas `consumer.apply(...).forRoutes(...)`. Los middleware Nest **no** se listan dentro de `providers[]` del mismo modo que un service: aplican mediante `MiddlewareConsumer`.

Paso 5 de 9 · app.module.ts ejemplo con imports y orden

Importá tus dos clases desde la ruta relativa correcta (desde donde está tu `app.module.ts`, típicamente algo como abajo si ambos están en `src/common/middlewares/`). Dentro de `apply()` el orden importa como lista: primer argumento ejecuta antes — acá necesitamos **Logger primero · Timing después**, igual que marca la consigna.

```
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { AppController } from '../app.controller';
import { AppService } from '../app.service';
import { LoggerMiddleware } from '../common/middlewares/logger.middleware';
import { TimingMiddleware } from '../common/middlewares/timing.middleware';

@Module({
  imports: [ /* ConfigModule.forRoot / TypeOrm si ya están en tu plantilla */ ],
  controllers: [AppController],
```

```
providers: [AppService],
})
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer): void {
    consumer.apply(LoggerMiddleware, TimingMiddleware).forRoutes('*');
  }
}
```

Botón del pánico · AppModule

Error típico: Cannot find module './common/middlewares/...' o clase no existe.

Acción: compruébala ruta física archivo vs import; export `class` con mismo nombre CamelCase; ejecutá build — TypeScript marca la línea exacta.

Paso 6 de 9 · Orden de ejecución

Si ves tiempos raros o logs duplicados, revisá orden en `apply()` y que no invoques `next()` dos veces en el mismo flujo accidentalmente.

Paso 7 de 9 · Compilación

En la raíz del proyecto:

```
npm run build
```

Paso 8 de 9 · Servidor desarrollo

```
npm run start:dev
```

Paso 9 de 9 · Probar con curl real

Adaptá la ruta a algo que exista (ejemplo `/products`).

```
curl -i http://localhost:3000/products
```

Verde: línea nueva en consola Nest + algo como `X-Response-Time: ... ms` entre cabeceras.

Checkpoint · laboratorio 1 cerrado

- Sin mirar archivo: ¿por qué el timing envuelve `res.send` y no espera sólo hasta `next()`?
- Si omitís el decorador `@Injectable()` en clase middleware ¿qué clase de problema esperarías antes de ejecutar?

- ¿Por qué `@Module()` con providers suele estar mal lugar para estos dos middleware específicos de lab?

Si estas tres te traban — volvé al diagrama inicial y dibujalo en papel con flechas antes de pegar código nuevo.

Parte III · capítulo B · TypeORM · bcrypt antes del token

Laboratorio 2 · Modelo usuario · hash · rutas públicas

Antes: teoría usuarios (`#usr-teoria`) · checklist `#usr-consigna` . Cada fichero lleva párrafos cortos tipo "esto hace falta porque..." más el contenido típico de referencia — ubicá estos trozos dentro de tus `*Service` / `*Module` ya existentes cuando el proyecto guía diverja en rutas pero no en contrato.

Paso 1 de 13 · Dependencias NPM

Desde carpeta proyecto (donde está `package.json`): instalá hashing y soporte típico SQLite/TypeORM ya usado en clase.

```
npm i bcrypt typeorm sqlite3 better-sqlite3
npm i -D @types/bcrypt
```

Lo crítico: que la instalación de **bcrypt nativo** complete sin errores en tu sistema operativo; si aparece problema de compilación, resolvé eso antes de seguir porque el hashing no llegará desde Node.

Paso 2 de 13 · `.env` ignorado por Git

```
.env
.env.*
!.env.example
```

Paso 3 de 13 · `.env.example` público · copia local privada

```
BCRYPT_COST=12
SQLITE_DATABASE=./database.sqlite
JWT_SECRET=cambia_esto_por_algo_largo_aleatorio_cuando_wirees_jwt
JWT_EXPIRES_SEC=3600
```

```
cp .env.example .env
```

Paso 4 de 13 · `ConfigModule` global en `AppModule`

Carga variables desde `.env` antes de construir `JwtModule` más adelante. `isGlobal: true` permite inyectar `ConfigService` en cualquier módulo sin reimportarlo siempre como dependencia repetida

literal.

```
ConfigModule.forRoot({
  isGlobal: true,
  envFilePath: ['.env', '.env.local'],
});
```

Paso 5 de 13 · Enum de roles

Este enum fuerza valores canónicos: la base sólo puede guardar estos strings y TypeScript atrapa errores antes de runtime.

```
export enum UserRole {
  USER = 'user',
  ADMIN = 'admin',
}
```

Paso 6 de 13 · Entidad `UserEntity` (archivo ejemplo)

`src/users/user.entity.ts`. Resumen columnaxcolumna: `id` UUID autogenerado; `email` marcado único para impedir duplicates; el hash de contraseña con `select: false` porque **casi todas** las lecturas debe traer sólo usuario “públicos” sin fila interna secreta hasta login explícito; campo `role` texto con default rol usuario porque la mayor parte de registros será user regular.

```
import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';
import { UserRole } from '../user-role.enum';

@Entity('users')
export class UserEntity {
  @PrimaryGeneratedColumn('uuid')
  id!: string;

  @Column({ unique: true })
  email!: string;

  @Column({ select: false })
  passwordHash!: string;

  @Column({ type: 'text', default: UserRole.USER })
  role!: UserRole;
}
```

Botón del pánico · TypeORM

Error típico: `Nest can't resolve Repository<UserEntity>.`

Acción rápida: tus `TypeOrmModule.forFeature([UserEntity])` debe vivir dentro del mismo módulo que declara `UserService`; revisá también `TypeOrmModule.forRoot / synchronize entities` cargando clase entidad igual que entrega template.

Paso 7 de 13 · Registrar entidad donde corresponda

Dentro del “users module”: importará sólo esa entidad vía pasarela oficial — eso permite inyección de repositorio correcta.

```
imports: [  
  TypeOrmModule.forFeature([UserEntity]),  
],
```

Además, el array central `TypeOrmModule.forRoot({ entities: [...] })` (o `synchronize pattern`) debe listar igual la clase porque TypeORM debe conocer modelo para crear tabla SQLite.

Paso 8 de 13 · Registro público (`register`) · hashing + rol inicial

Servicios Nest son `@Injectable()`; inyectás repos y `ConfigService`. Flujo habitual: leemos cost bcrypt desde variables; calculamos número entero seguridad fallback 12 si falta texto; ejecutamos bcrypt; contamos registros tabla — si primera fila será admin clase guía cerrada — después insertamos nueva fila con role calculado servidor (cliente NUNCA manda campo role público).

```
const rounds = Number(cfg.get<string>('BCRYPT_COST') ?? '12');  
const passwordHash = await bcrypt.hash(plainPassword, rounds);  
  
const countUsers = await this.usersRepo.count();  
const role =  
  countUsers === 0 ? UserRole.ADMIN : UserRole.USER;  
  
const entity = this.usersRepo.create({  
  email: dto.email.trim().toLowerCase(),  
  passwordHash,  
  role,  
});  
await this.usersRepo.save(entity);
```

Paso 9 de 13 · Login — traer hash + `bcrypt.compare`

Como `passwordHash` está oculto por defecto, cargás resultado explícitamente con selección incluido hash (patrón básico usando querybuilder). Después mismo mensaje público ante credencial inválido para no filtrar listado correos válidos.

```
// imports típicos: bcrypt, UnauthorizedException, repository..  
const email = dto.email.trim().toLowerCase();  
const q = this.usersRepo
```

```

    .createQueryBuilder('u')
    .addSelect('u.passwordHash')
    .where('u.email = :email', { email });
const user = await q.getOne();

if (!user) {
  throw new UnauthorizedException('Credenciales inválidas');
}

const ok = await bcrypt.compare(dto.password, user.passwordHash);
if (!ok) {
  throw new UnauthorizedException('Credenciales inválidas');
}

```

Paso 10 de 13 · Mensaje neutro ante fallo login

Usamos `UnauthorizedException` desde `@nestjs/common` repetida arriba: así no distinguís causa frente público accidental.

Paso 11 de 13 · Build intermedio

```
npm run build
```

Paso 12 de 13 · curl rápido

```

curl -s -X POST http://localhost:3000/auth/register \
-H 'Content-Type: application/json' \
-d '{"email":"primer@algo","password":"Clave_segura_demo_9"}'

```

Paso 13 de 13 · Segundo usuario

Otro `/auth/register` ⇒ verificás que llega rol `user` salvo modelo del curso especifique otra bootstrap rule.

Checkpoint · laboratorio 2 · sin saltar

- ¿Por qué si quitamos `@Injectable()` del servicio de usuarios el sistema deja de armar dependencias correctamente?
- ¿Qué ataque reduce el mensaje homogéneo "Credenciales inválidas" comparado con "Email no registrado"?
- Explicá en una frase por qué `select: false` en `passwordHash` te obliga a un truco extra en login.

Parte III · capítulo C · JWT, Passport, roles, curls finales

Laboratorio 3 · De contraseña válida a Bearer token con admin panel

Prerrequisito duro: labs A+B compilan · podés registrar/login sin Bearer. Abajo encontrás párrafos que explican rol de cada clase + archivos típicos cerrados funcionando coherentes entre sí (adaptá rutas import exactas igual como en tu proyecto guía `api-c`).

1. **NPM:** `@nestjs/jwt` · Passport · `passport-jwt` (+ types dev donde falten).
2. Decorador `@Roles()`, `RolesGuard` en providers del módulo correcto.
3. `JwtStrategy` mismo nombre secreto/string estrategia `'jwt'`.
4. `JwtModule.registerAsync` leyendo `JWT_SECRET` / `JWT_EXPIRES_SEC`.
5. Guards ordenados sobre controladores: autenticación primero · autorización segundo.

1 Paquetes (desde carpeta proyecto):

```
npm i @nestjs/jwt @nestjs/passport passport passport-jwt
npm i -D @types/passport-jwt
```

2 Decorador `@Roles()` — marca metadata leída luego por `RolesGuard`; no es el guard mismo. Archivo habitual `src/common/decorators/roles.decorator.ts`:

```
import { SetMetadata } from '@nestjs/common';
import { UserRole } from '../../../users/user-role.enum';

export const ROLES_KEY = 'roles';
export const Roles = (...roles: UserRole[]) =>
  SetMetadata(ROLES_KEY, roles);
```

3 `RolesGuard` completo:

Este guard pregunta: “¿el handler exige roles declarados mediante decoradores? Si sí, ¿los que vienen después de `JwtAuth` cumplen?” Cuando nadie marcó ningún rol, retorna verdadero porque entonces ese aspecto seguridad sólo-role no aplicaba.

```
import { CanActivate, ExecutionContext, Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { ROLES_KEY } from '../../../decorators/roles.decorator';
import { UserRole } from '../../../users/user-role.enum';

@Injectable()
export class RolesGuard implements CanActivate {
```

```

constructor(private readonly reflector: Reflector) {}

canActivate(ctx: ExecutionContext): boolean {
  const required =
    this.reflector.getAllAndOverride<UserRole[] | undefined>(ROLES_KEY, [
      ctx.getHandler(),
      ctx.getClass(),
    ]);
  if (!required?.length) return true;

  const req = ctx.switchToHttp().getRequest();
  const role = req.user?.role as UserRole | undefined;
  return !!role && required.includes(role);
}
}

```

Botón del pánico · RolesGuard

Error típico: `Can't resolve dependencies of RolesGuard (?)`.

Solución: agregá `RolesGuard` y `Reflector` ya está global en core — normalmente falta declarar `RolesGuard` como `provider` del módulo donde usás controladores que lo referencian.

4 Coexistencia módulos: `TypeOrmModule.forFeature()` y `JwtModule.registerAsync()` conviven en `imports` del respectivo módulo; cada provider necesita su módulo fuente importado.

5 JwtAuthGuard (shim Passport): hereda `AuthGuard('jwt')` del string idéntico usado al registrar `PassportStrategy(Strategy, 'jwt')`. No requiere lógica extra en clase porque Passport internamente ejecuta estrategias ya registradas.

```

import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}

```

6 JwtStrategy — le extrae Bearer, verifica contra secreto igual que tus firmaciones login. En `validate()` cargamos usuario fresco desde BD buscando `payload.sub`: ese objeto resultado es lo que aparece después como `req.user` para guards posteriores.

```

import { Injectable } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { PassportStrategy } from '@nestjs/passport';
import { InjectRepository } from '@nestjs/typeorm';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { Repository } from 'typeorm';
import { UserEntity } from '../users/user.entity';

```

```

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy, 'jwt') {
  constructor(
    cfg: ConfigService,
    @InjectRepository(UserEntity)
    private readonly usersRepo: Repository<UserEntity>,
  ) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: cfg.getOrThrow<string>('JWT_SECRET'),
    });
  }

  async validate(payload: { sub: string }) {
    const user = await this.usersRepo.findOne({ where: { id: payload.sub } });
    if (!user) {
      return null;
    }
    return { id: user.id, role: user.role, email: user.email };
  }
}

```

Botón del pánico · JwtStrategy · DI

Error típico: Nest can't resolve dependencies of JwtStrategy (?).

Checklist rápido: ¿ `UserEntity` está dentro `TypeOrmModule.forFeature` mismo módulo que declara `JwtStrategy`?

Otro: ¿ `@nestjs/passport` / `TypeOrmModule` efectivamente están en imports de `AuthModule` antes de ejecutar compilación?

7

AuthModule ejemplo — registra Jwt + PassportStrategy provider: Nota sintaxis: vos agregás arriba decorador `@Module({ ... })` y los `import` necesarios desde los paquetes Nest y TypeORM.

```

@Module({
  imports: [
    ConfigModule,
    PassportModule.register({ defaultStrategy: 'jwt' }),
    JwtModule.registerAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (cfg: ConfigService) => ({
        secret: cfg.getOrThrow<string>('JWT_SECRET'),
        signOptions: {
          expiresIn: Number(cfg.get<string>('JWT_EXPIRES_SEC')) ?? '3600',

```

```

    },
  }),
}),
TypeOrmModule.forFeature([UserEntity]),
],
providers: [
  AuthService,
  JwtStrategy,
  JwtAuthGuard,
  RolesGuard,
],
exports: [
  JwtModule,
  JwtAuthGuard,
  RolesGuard,
],
})
export class AuthModule {}

```

Completá imports reales de tu proyecto (`AuthService` , controladores exportados donde corresponda); esta pieza resume qué debe coexistir dentro del mismo `AuthModule` porque Passport y TypeORM tienen prerequisites importados antes de ejecutar tus providers especializados (`JwtStrategy` ejemplo).

8 AuthService — firmar después de bcrypt ok login:

Una vez verificadas credenciales, armás objeto payload mínimo coherente con lo que después leerán guards: típicamente `sub` id usuario texto y campo `role` porque RolesGuard después lo encuentra dentro `req.user` ya hidratado vía JwtStrategy validate.

```

const accessToken = this.jwtService.sign({
  sub: user.id,
  role: user.role,
});
return { access_token: accessToken };

```

Las rutas `/auth/register` públicas siguen reglas proyecto (no exponer hash; no aceptar role arbitrario cliente).

9 Controllers admin (orden guards):

```

@Get('users')
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(UserRole.ADMIN)
findAllUsers() {
  ...
}

```

10 Pruebas con terminal (servidor vivo):

```
# 0 • Servidor npm run start:dev • .env con secretos válidos cargados • no
subas ese archivo a Git.

# Opcional limpio: reproducir primera fila admin desde SQLite vacío
# rm ./database.sqlite # sólo dev acordado con docente!

curl -s -X POST localhost:3000/auth/register \
  -H 'Content-Type: application/json' \
  -d
'{"email":"bootstrap@demo","password":"Clave_demo_larga_para_hash_abc123"}'

curl -s -X POST localhost:3000/auth/login \
  -H 'Content-Type: application/json' \
  -d
'{"email":"bootstrap@demo","password":"Clave_demo_larga_para_hash_abc123"}'

TOKEN='PEGÁ_access_token_sin_comillas_rotas'

curl -si localhost:3000/users \
  -H "Authorization: Bearer $TOKEN"

curl -s localhost:3000/placeholder-users
```

Reglas modelo curso cerrado: primer usuario ADMIN cuando BD vacía; después USER hasta promoción admin; no abuses endpoints públicos de rol si tu repo ya los cerró igual guía `api-c`.

Checkpoint • laboratorio 3 cerrado • JWT vivo

- Misma pregunta dura clase: ¿qué haría RolesGuard antes de JwtAuthGuard en handler admin y por qué duele tanto?
- ¿Qué garantía perdés si sólo mirás JWT decodificado en jwt.io pero no ejecutás curls sobre tu API?
- Una línea definiendo cuando devolvés HTTP 401 frente HTTP 403 con guardias Nest-Passport en este proyecto.

Antes de pedir ayuda gritando

Error que aparece antes que todo en Nest (~90% clase práctica backend)

Mensaje: Nest can't resolve dependencies of TuClase (?). Please make sure...

Traducción humana: Nest necesitaba crear TuClase pero no encuentra algo que deberías haber dicho cómo obtener (otro provider, módulo import, token literal equivocado).

Hoja de ruta de 45 segundos:

1. ¿La clase aparece como **providers** del módulo donde se usa?
2. ¿Los módulos que exportan ese provider están en **imports**?
3. Si usa TypeORM: ¿hay **TypeOrmModule.forFeature([Entidad])** en ese módulo?

- **¿Pongo la validación JWT en un middleware global?** Preferí guards: conocen el handler, respetan `@Public()` vía reflexión y ordenan mejor el stack JWT → roles sin hacks de enrutado.
- **req.user undefined después del guard.** Revisa que la estrategia devuelva objeto con los campos que esperás; que el módulo importe Passport y que el mismo string de estrategia use guard y estrategia (p. ej. `'jwt'`).
- **401 vs 403 mezclados.** Sin token o token inválido ⇒ no demostrás identidad (401 en flujos típicos Passport). Token válido pero rol insuficiente ⇒ autorización denegada (403).
- **Olvidar el orden:** primero autenticación (`JwtAuthGuard`), después autorización (`RolesGuard`). Evaluar roles sin usuario cargado sólo produce frustración silenciosa o errores genéricos.
- **Confiar en el cliente para el campo `role`.** El DTO público no debería aceptarlo; el servidor asigna `user` salvo bootstrap de primera fila vacía en BD.
- **Hashes "rápidos tipo SHA" sin costo alto.** Protección débil ante diccionario; bcrypt/Argon2 con factores configurables están en el bloque de teoría de usuarios / bcrypt.

Tabla corta middleware · guard · interceptor

Concepto	Cuándo	Ejemplos rápidos
Middleware	Early HTTP global / por prefijo / por controlador (config consumer).	Logger del lab, tiempo <code>X-Response-Time</code> .
Guard	¿Se ejecuta ese handler específico?.	<code>JwtAuthGuard</code> , <code>RolesGuard</code> .
Interceptor	Envolver handler con lógicas transversal basadas observable/promesa.	Formato estándar de respuesta, logging fino tras handler.

Siguientes pasos sugeridos

- Antes de refactorizar a ciegas: **FAQ** · checklist “Nest can't resolve” (`providers`, `imports`, `forFeature`, orden de guards) suele explicar guards vacíos o compilación rota.
- Documentación Nest oficial: Middleware, Guards, Interceptors, Authentication & Passport, TypeORM Recipes.
- Capa de datos: cuando `synchronize: true` dejó de bastar pasá a **migraciones declarativas** antes de exponer públicos tus endpoints.
- Seguridad adicional según modelo de amenaza: límites de petición / bloqueos por IP tras login repetido, revocaciones de JWT, segunda factor OTP.

Guía de supervivencia imprimible: **Partes I–III** como antes (teoría · consignas · pasos ejecutables con checkpoints). Abrió este HTML en el navegador: **Imprimir → Guardar como PDF** para llevar offline.